Practice · Functions

Each question lists its options; the right one is marked **Right answer**. *Feedback* is what students see after they answer. In a typed question the blank is in square brackets, with the accepted answers separated by `|`.

(mc) L1 · What the parameter is

```
int square(int x) {
    return x * x;
}
```

What is `x` in this function?

- The name of the function.
- **Right answer:** The parameter: a box that gets the value you put in the brackets when you call `square`.
- The answer the function gives back.
- A variable that `main` can use.

Feedback: The first line has the type of the answer (`int`), the name (`square`) and, in brackets, the parameter (`int x`). When you call `square(5)`, the 5 is copied into `x`. Class 7, slides 15 and 18.

(mc) L1 · Where the function goes

In this course, where do you write your own function?

- Inside `main`, before `return 0;`.
- Below `main`, at the end of the file.
- In a separate file.
- **Right answer:** Above `main`, after the `#include` lines.

Feedback: gcc reads the file from the top down, so it must meet the function before the call. Write the whole function above `main`. The other way is a prototype line above `main`, `int square(int x);`, with the function below. Class 7, slides 33 to 35.

(mc) L1 · What void means

What does `void` mean in `void greet(void)`?

- **Right answer:** The first `void`: the function gives nothing back. The second: it takes no values.
- The function is empty and does nothing.
- The function can only be called once.
- The function gives back 0.

Feedback: A `void` function does a job, like printing, and gives no answer back. `(void)` in the brackets means you call it with nothing inside: `greet();`. The brackets are still needed. Class 7, slides 43 and 44.

(mc) L2 · A call as a value

```
int add(int a, int b) {
    return a + b;
}

int main(void) {
    printf("%d\n", add(add(1, 2), 4));
```

```
    return 0;
}
```

What does it print?

- `3`
- **Right answer:** `7`
- `8`
- `12`

Feedback: Start inside: `add(1, 2)` gives back 3. Then `add(3, 4)` gives back 7. A call is replaced by its value, so a call can be the value you pass to another call. Class 7, slides 30 and 31.

(mc) L2 · A loop inside a function

```
int power(int base, int exp) {
    int result = 1;
    for (int i = 1; i <= exp; i++) {
        result *= base;
    }
    return result;
}
```

What is `power(2, 4)`?

- `8`
- `6`
- **Right answer:** `16`
- `32`

Feedback: The loop runs `exp` times, 4 times, and multiplies `result` by 2 each time: 2, 4, 8, 16. `result` starts at 1, like a factorial. Class 7, slides 38 to 40.

(mc) L2 · A call without brackets

```
void greet(void) {
    printf("Hi\n");
}

int main(void) {
    greet;
    greet();
    return 0;
}
```

You compile with `gcc -Wall` and run it. What happens?

- It does not compile.
- It prints `Hi` twice.
- **Right answer:** gcc warns `statement with no effect`, and the program prints `Hi` once.
- It prints nothing.

Feedback: `greet;` only names the function; it does not call it, so it does nothing, and `-Wall` says so. `greet();` with brackets is the call. That one prints `Hi`. Class 7, slide 79.

(fib) L3 · The type of the answer

`cube` must give back a whole number. Type the word that is missing at the start of the first line.

```
[int] cube(int x) {
    return x * x * x;
}
```

Feedback: A whole number is an `int`, so the function's first word is `int`. `cube(3)` then gives back 27. Class 7, slides 16 and 83.

(fib) L3 · The return line

Complete the `return` line so that `cube(3)` gives back 27. Type only what comes after `return`, without the semicolon.

```
int cube(int x) {
    return [x * x * x|x*x*x|x * x*x|x*x * x];
}
```

Feedback: The cube of `x` is `x` times `x` times `x`. `return` hands that value back to whoever called. Class 7, slides 83 and 85.

(fib) L3 · Keep the answer

Complete the line in `main` so that `y` gets the square of 5. Type only what comes after `=`, without the semicolon.

```
int square(int x) {
    return x * x;
}

int main(void) {
    int y = [square(5)|square( 5 )];
    printf("%d\n", y);
    return 0;
}
```

Feedback: The call is the name with the value in brackets: `square(5)`. It is replaced by 25, which lands in `y`, and the program prints `25`. Class 7, slide 20.